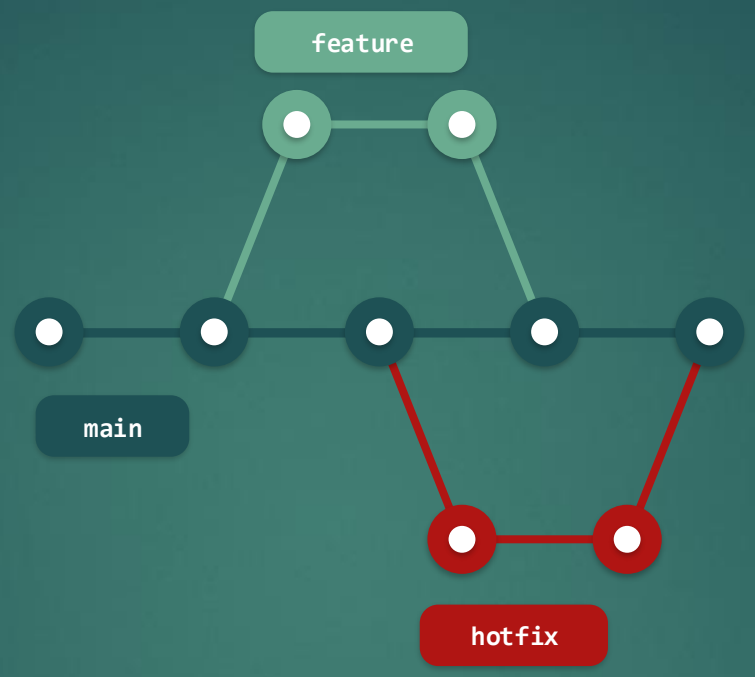


2. Version Control

LEIM – DAM

What is Version Control?

A system that records changes to files over time so you can recall specific versions later.

Key Benefits:

- **History tracking** – See who changed what and when
- **Collaboration** – Multiple people can work simultaneously
- **Backup & Recovery** – Restore any previous version
- **Accountability** – Full audit trail of every change
- **Branching** – Experiment without affecting the main project

Types of Version Control

Centralized (CVCS)

- A single central server stores the repository and its history
- Developers commit changes directly to the central server
- Most operations require a network connection
- Single point of failure (if the server goes down, collaboration stops)
- Examples: SVN, CVS, Perforce

Distributed (DVCS)

- Each developer has a complete copy of the repository (including history)
- Most operations can be performed offline
- No single point of failure (each clone acts as a backup)
- Changes are shared via push/pull between repositories
- Examples: Git, Mercurial

What is Git?

Git is a free, open-source distributed version control system created by Linus Torvalds in 2005 for Linux kernel development.

Key Characteristics:

- **Speed** – Most operations are local and nearly instant
- **Integrity** – Every file is checksummed (SHA-1 hash)
- **Snapshots** – Stores snapshots, not file differences
- **Branching** – Lightweight branching and merging

Git Setup – First Commands

Configure your identity:

```
git config --global user.name "Your Name"
git config --global user.email you@example.com
```

Create a new repository:

```
git init my-project
```

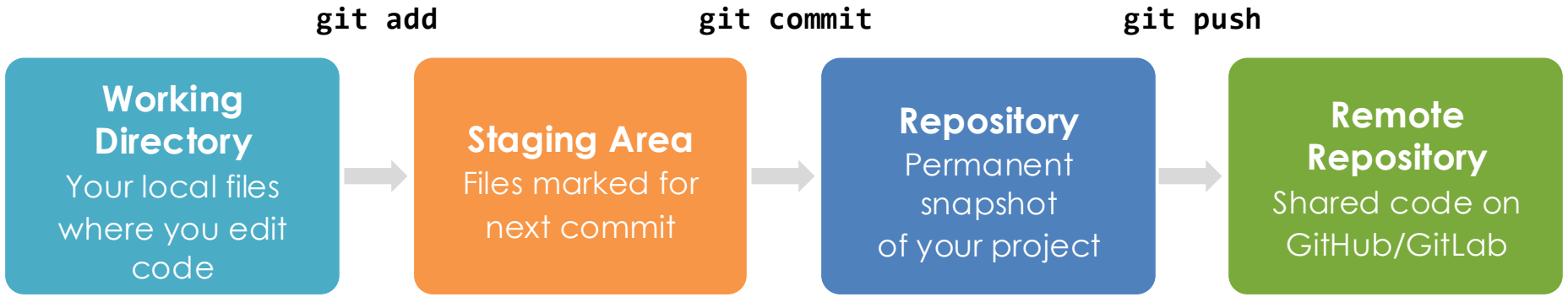
Creates a new .git directory to track changes

Clone an existing repository:

```
git clone https://github.com/user/repo.git
```

Downloads a full copy of a remote repository

The Git Workflow – Four Areas



Files move from Working Directory → Staging Area → Repository → Remote
This multi-step process gives you full control over what goes into each commit and push.

Tracking Changes

Check status of your files:

`git status`	Shows modified, staged, and untracked files

Stage files for commit:

`git add filename.txt`	Stage a single file
`git add .`	Stage all changes

Save a snapshot (commit):

```
git commit -m "Add login feature"
```

Viewing History

View commit history:

`git log`	Full history with details
`git log –oneline`	Compact one-line format
`git log –graph`	Visual branch graph

Compare changes:

`git diff`	Unstaged changes
`git diff --staged`	Staged changes
`git show abc1234`	Show a specific commit

Branching & Merging

Branches let you develop features in isolation without affecting the main codebase.

Branch commands:

`git branch`	List all local branches
`git branch feature/name`	Create a new branch
`git switch feature/name`	Switch to an existing branch
`git switch -c feature/name`	Create and switch to a new branch

Merging (switch to main branch and do the merge):

`git switch main`	Switch to main branch
`git merge feature/name`	Merge branch into main

feature/name is the branch name
feature means new feature

Examples:

- feature/user-profile
- bugfix/login-error
- hotfix/payment-failure
- refactor/api-client

Remote Repositories

Remotes are versions of your repository hosted on a server (GitHub, GitLab, etc.).

Managing remotes:

`git remote -v`	List remote connections
`git remote add origin <url>`	Add a new remote repository

Syncing with remotes:

`git push origin main`	Upload local commits to remote
`git pull origin main`	Fetch and merge changes from remote
`git fetch origin`	Download changes without merging

Tip: git pull = git fetch + git merge

Undoing Changes

Discard uncommitted changes:

```
git restore filename.txt
```

Restores file to the last committed version

Unstage a file:

```
git restore --staged filename.txt
```

Removes file from staging area (keeps changes)

Undo a commit (safe for shared repos):

```
git revert abc1234
```

Creates a new commit that undoes the specified commit

Reset to a previous state (use with caution!):

`git reset --hard HEAD~1` Go back 1 commit

Practical Example – Full Workflow

Scenario: You need to add a login feature to a project.

```
git clone https://github.com/team/project.git

cd project
git checkout -b feature-login ← Create branch

# ... edit login.py ...
git add login.py          ← Stage changes

git commit -m "Add login page" ← Commit
git push origin feature-login ← Push to remote
git checkout main         ← Switch to main

git merge feature-login       ← Merge feature
```

Git Best Practices

Write clear commit messages:

- ✅ `"Fix null pointer in user authentication"`
- ❌ `"Fixed stuff”`

Commit often, push regularly:

- Small, focused commits are easier to review and revert

Use branches for features:

- Never commit directly to main/master

Use .gitignore:

- Exclude build files, secrets, IDE configs, node_modules/

Pull before you push:

- Always `git pull` before pushing to avoid merge conflicts

Git Commands – Cheat Sheet

Category	Command	Description
Repository setup	git init	Initialize a new repository
	git clone <url>	Clone an existing repository
Checking status	git status	Show changed files
	git log	Show commit history
Staging	git add <file>	Stage a file
	git add .	Stage all changes
Commit	git commit -m "message"	Commit staged changes
Branching	git branch	List branches
	git branch feature/x	Create a new branch
	git switch feature/x	Switch to an existing branch
	git switch -c feature/x	Create and switch to a branch
Merging	git switch main	Switch to main branch
	git merge feature/x	Merge branch into current branch
Remotes	git remote -v	Show remote repositories
	git remote add origin <url>	Add a remote repository
Synchronizing	git push origin main	Upload commits to remote
	git pull origin main	Fetch and merge changes
	git fetch origin	Download changes without merging